



LeetCode 451: Sort Characters By Frequency

1. Problem Title & Link

Title: LeetCode 451: Sort Characters By Frequency

Link: <https://leetcode.com/problems/sort-characters-by-frequency/>

2. Problem Statement (Short Summary)

Given a string *s*, sort the characters in **decreasing order of frequency**.

Characters that appear more times should come first.

If a character appears *k* times, it should appear *k* times consecutively in the result.

Example

Input:

s = "tree"

Output:

"eert"

or

"eetr"

Explanation:

e appears 2 times

t appears 1 time

r appears 1 time

Since e appears most frequently, it comes first.

3. Examples (Input → Output)

Example 1

Input:

s = "tree"

Output:

"eert"

Example 2

Input:

s = "cccaaa"

Output:

"cccaaa"

or

"aaaccc"



Both are valid.

Example 3

Input:

`s = "Aabb"`

Output:

`"bbAa"`

or

`"bbaA"`

Explanation:

b -> 2 times

A -> 1 time

a -> 1 time

Remember:

`'A' != 'a'`

Case-sensitive.

4. Constraints

$1 \leq s.length \leq 5 \times 10^5$

s contains:

English letters

digits

symbols

Important:

Sort by frequency descending.

5. Core Concept (Pattern / Topic)

HashMap / Counting

Secondary Topics:

- Bucket Sort
- Custom Sorting

6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

Input:

`"tree"`



Count frequencies:

t → 1

r → 1

e → 2

Now sort by frequency:

e → 2

t → 1

r → 1

Build result:

"eert"

Brute Force Idea

For every character:

Count its occurrences by scanning the string again.

This would take:

$O(n^2)$

Too slow.

Better Observation

We only need:

Frequency of each character

A HashMap is perfect.

Step 1: Count Frequencies

Example:

s = "tree"

HashMap:

```
{  
  t:1,  
  r:1,  
  e:2  
}
```

Step 2: Sort by Frequency

Sort entries using:



frequency descending

Result:

e -> 2

t -> 1

r -> 1

Step 3: Build Answer

e repeated 2 times = "ee"

t repeated 1 time = "t"

r repeated 1 time = "r"

Answer:

"eetr"

or

"eert"

Both valid.

Key Insight

This problem is:

Count Frequencies

+

Sort by Frequency

A very common interview pattern.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
tree
Frequency Map:
t -> 1
r -> 1
e -> 2
Sort:
e -> 2
t -> 1
r -> 1
Build:
ee
```



```
+
t
+
r
Result:
"eetr"
```

8. Pseudocode (Language Independent)

```
count frequencies

sort characters by frequency descending

result = ""

for each character
    append character
    frequency times

return result
```

9. Code Implementation

Python

```
from collections import Counter

class Solution:
    def frequencySort(self, s: str) -> str:

        freq = Counter(s)

        chars = sorted(
            freq.keys(),
            key=lambda ch: freq[ch],
            reverse=True
        )

        result = []

        for ch in chars:
            result.append(ch * freq[ch])

        return "".join(result)
```

Java

```
import java.util.*;
```



```
class Solution {  
  
    public String frequencySort(String s) {  
  
        HashMap<Character,Integer> map =  
            new HashMap<>();  
  
        for(char ch : s.toCharArray()) {  
  
            map.put(  
                ch,  
                map.getOrDefault(ch,0)+1  
            );  
        }  
  
        List<Character> chars =  
            new ArrayList<>(map.keySet());  
  
        chars.sort(  
            (a,b) ->  
                map.get(b) - map.get(a)  
        );  
  
        StringBuilder result =  
            new StringBuilder();  
  
        for(char ch : chars) {  
  
            int count = map.get(ch);  
  
            while(count-- > 0) {  
  
                result.append(ch);  
            }  
        }  
  
        return result.toString();  
    }  
}
```

10. Time & Space Complexity

Time Complexity

Frequency counting:

$O(n)$

Sorting unique characters:

$O(k \log k)$



where:

k = unique characters

Overall:

$O(n + k \log k)$

Space Complexity

$O(k)$

For frequency map.

11. Common Mistakes / Edge Cases

Mistake 1

Sorting alphabetically.

Example:

tree

Alphabetical:

eert

may accidentally work here.

But:

cccaaa

must be sorted by frequency, not alphabet.

Mistake 2

Not repeating characters.

Wrong:

etr

Need:

eetr

Mistake 3

Ignoring case sensitivity.

Example:

Aabb

$A \neq a$

Edge Cases

Single Character

Input:



"a"

Output:

"a"

All Same Character

Input:

"aaaa"

Output:

"aaaa"

Digits & Symbols

Input:

"111!!"

Output:

"111!!"

or

"!!111"

depending on frequency ties.

12. Detailed Dry Run

Input:

s = "tree"

Frequency Map:

Character	Frequency
t	1
r	1
e	2

Sorted Order:

Character	Frequency
e	2
t	1
r	1

Build:

"ee"

+



"t"

+

"r"

Result:

"eetr"

13. Common Use Cases (Real-Life / Interview)

Text Analytics

Most frequent characters.

Log Analysis

Most common events.

Data Compression

Frequency-based encoding.

Interview Purpose

Tests:

HashMap

Frequency Counting

Custom Sorting

14. Common Traps (Important!)

Trap 1

Sorting characters instead of frequencies.

Trap 2

Forgetting repeated occurrences.

Trap 3

Assuming tied frequencies require alphabetical order.

Problem doesn't require that.

15. Builds To (Related LeetCode Problems)

LC 451 - Sort Characters By Frequency



LC 242 - Valid Anagram



LC 49 - Group Anagrams



LC 347 - Top K Frequent Elements



LC 692 - Top K Frequent Words

These strengthen frequency-counting patterns.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
HashMap + Sort	$O(n + k \log k)$	$O(k)$	Best
Bucket Sort	$O(n)$	$O(n)$	Advanced
Priority Queue	$O(n \log k)$	$O(k)$	Good

Approach 1: HashMap + Sort (Chosen)

Most intuitive.

Most common interview answer.

Approach 2: Bucket Sort

Create buckets by frequency.

Can achieve:

$O(n)$

time.

Approach 3: Max Heap

Store:

(frequency, character)

Useful for Top-K style problems.

17. Why This Solution Works (Short Intuition)

A frequency map tells us how many times each character appears.

Sorting characters by frequency and repeating each character according to its count produces exactly the required order.

18. Variations / Follow-Up Questions

Follow-Up 1



Sort words by frequency instead of characters.

Related:

LC 692

Top K Frequent Words

Follow-Up 2

Return only the top K frequent characters.

Follow-Up 3

Sort by frequency ascending.

Follow-Up 4

Group characters with the same frequency together.

Interview Takeaway

The key pattern is:

HashMap

+

Frequency Counting

+

Custom Sorting

Whenever you hear:

Most Frequent

Frequency Ranking

Count Occurrences

Top K

think:

HashMap

first. Then decide whether you need:

Sorting

Heap

Bucket Sort

to organize the frequencies. This pattern appears repeatedly in string, array, and data-analysis interview questions.